

Agent-Runtime Security for Kubescape — Posture, Admission, and gVisor Runtime Visibility

Status: Draft for discussion

Author: Yash Gupta (yellow-forrest)

Reviewers: @matthyx · @slashben

Last updated: 2026-08-18

Status	Draft for discussion
Primary Author	Yash Gupta (yellow-forrest)
Collaborators	—
Reviewers	@matthyx (Matthias Bertschy), @slashben (Ben Hirschberg) — Kubescape maintainers
Upstream tracking	kubescape/kubescape#2557 · design designs-and-proposals#10 · cncf/mentoring#1991 · this RFC upstream: regolibrary#789 , designs-and-proposals#14
Companion PoC	gVisor seccheck collector — cluster-free E2E green (8/8 points), benchmarked (this doc)
Last updated	2026-08-18

Approvers

Name	yes	not yet	Date
@matthyx			
@slashben			

Table of Contents

1. [Overview](#)
2. [Motivation](#)
3. [Requirements](#)
4. [Background](#)
5. [Goals and Non-Goals](#)
6. [Product & User Use Cases](#)
7. [Gap Analysis — what is and isn't captured upstream](#)
8. [Competitive & Vendor Landscape — what security companies target](#)
9. [Design](#)
10. [Benchmarks & Performance](#)

- | | |
|---|---|
| 11. Resource & Telemetry Cost Model | 16. Security / Privacy / Compliance |
| 12. Timeline | 17. Risks |
| 13. Dependencies | 18. Future Work |
| 14. Alternatives Considered / Prior Art | 19. Audit Log |
| 15. Operations | 20. References |

Overview

TLDR. GKE Agent Sandbox went GA in May 2026 and, with Agent Substrate, pushes the security-relevant configuration of AI agents into new custom resources (`Sandbox` / `SandboxTemplate` under `agents.x-k8s.io` ; `WorkerPool` / `ActorTemplate` under `ate.dev`) that live outside the workloads Kubescape scans today. This RFC proposes two complementary workstreams:

- **Part A — static posture + admission.** A new *Agent Runtime Hardening* Rego framework that treats the agent CRDs as first-class scan targets (isolation class, default-deny egress, per-actor resource ceilings, image provenance, per-session credential scoping, snapshot encryption), surfaced through the existing CLI / operator / MCP paths and an admission (VAP) variant, mapped to OWASP-LLM-2025 and MITRE ATLAS. This rides entirely on capability Kubescape already has.
- **Part B — runtime visibility.** A node-agent sensor that closes the gVisor blind spot: because a gVisor actor's syscalls are serviced by the userspace Sentry and never reach the host kernel, the node-agent's host-eBPF probes cannot see inside the sandbox. A **working proof-of-concept** already speaks gVisor's `seccheck` remote-sink protocol, decodes the real trace-point stream, and normalizes each point into the node-agent event shape the detection engine already consumes — so existing runtime rules fire on sandboxed actors unchanged.

The underlying gaps are already recognized upstream — the blind spot, the two-part split, the candidate signal sources (node-agent#894), and the mapping goals are all on record. What this RFC adds is making them **executable**: a validated signal source, a concrete node-agent integration seam, and the operational machinery to run it safely despite gVisor's session constraints. Part A ships value in a quarter and blocks on nothing. Part B is the harder, higher-value half and is already de-risked by the PoC. Measured PoC decode throughput is **≈755K events/second/core** at **~130 bytes/event** on the wire; the end-to-end incremental infrastructure cost is dominated not by compute (≈0.1% of a node) but by **telemetry egress**, which is the central architectural argument for node-local profiling before export.

Motivation

Why now. Google made **GKE Agent Sandbox GA in May 2026** and introduced **Agent Substrate** alongside it. Both concede that the Kubernetes control plane was not built for how agents behave, and both move the security-

relevant configuration into new object types. Adoption is not hypothetical: Google reports **16× growth in five months**, sandbox allocation at **300 sandboxes/second/cluster**, and **sub-second** sandbox startup (p90 ≈ 200 ms). The workload inside is untrusted *by definition* — it is model-generated code executed at runtime — so the isolation and egress posture of the resource that defines it is the whole trust boundary.

Two facts make this a security gap rather than a configuration nicety:

- 1. **The definitions are unscanned.** An `ActorTemplate` decides whether model-generated code runs under gVisor/Kata or a shared kernel; a `WorkerPool` decides whether one runaway actor can starve every other session multiplexed onto it. Nothing in Kubescape treats these as posture decisions today.
- 2. **The runtime is opaque.** Even a perfectly hardened sandbox is a black box to the node-agent once it is running. gVisor's own maintainers and Kubescape's parent company both state the split plainly: the sandbox "controls *where* the agent runs ... it says nothing about *what* the agent does once running." Static posture says *the door is locked*; runtime visibility says *someone is climbing through the window anyway*.

This RFC closes both, in the codebase (Kubescape scan/admission + node-agent) that already has the machinery to carry them.

Requirements

Functional requirements

#	Requirement	Part
FR-1	Scan <code>Sandbox</code> / <code>SandboxTemplate</code> (<code>agents.x-k8s.io</code> , <code>extensions.agents.x-k8s.io</code>) and <code>WorkerPool</code> / <code>ActorTemplate</code> / <code>SandboxConfig</code> (<code>ate.dev</code>) as native resources.	A
FR-2	Ship a named <i>Agent Runtime Hardening</i> framework wiring the already-merged agent controls (C-0297 hardened runtime class, C-0301 agent egress) plus new controls, mapped to OWASP-LLM-2025 and MITRE ATLAS.	A
FR-3	Surface findings through the existing CLI, operator, and MCP-server paths with Kubescape-style remediation.	A
FR-4	Provide an admission variant (VAP / Kubescape admission) rejecting or warning on non-hardened agent definitions, with an explicit note that admission fires on definition CREATE/UPDATE, not on out-of-band actor spawns.	A
FR-5	Recover runtime signals from inside a gVisor sandbox (exec, network, open, fork, exit, ptrace) and normalize them into node-agent event types.	B
FR-6	Attribute each recovered event to its sandbox/container id for workload correlation.	B

#	Requirement	Part
FR-7	Detect <code>Default</code> -session contention and degrade gracefully to host-boundary signals plus a posture finding, rather than going dark.	B

Non-functional requirements

#	Requirement	Target (from measurement / published data)
NFR-1	Decode/normalize CPU overhead per node	≤ 1% of one core at high load (measured: 0.12% of an 8-vCPU node at 7,500 ev/s)
NFR-2	Sensor memory footprint	Bounded; all wire fields size-capped (the sandboxed workload is the adversary)
NFR-3	No added latency on the actor's hot path	Sensor is out-of-band (Sentry → UDS → collector); zero inline blocking
NFR-4	Scan-path parity	Rego controls validate offline against fixtures with regolibary's testrunner (no cluster)
NFR-5	Telemetry cost	Node-local profiling before export; ≤ ~5% of raw events persisted (cost model §)
NFR-6	CRD schema resilience	Field access behind a thin helper for the still-drifting <code>ate.dev/v1alpha1</code> group

Product requirements: this RFC is the technical design; the product scope is the LFX Term-3 mentorship deliverable set (Sep–Nov 2026). Part A is fully in scope; Part B lands the sensor + node-agent seam in scope, with full multi-sandbox brokering and GKE-managed-session interaction as term-time stretch.

Background

Key terms.

- **GKE Agent Sandbox / Agent Substrate** — Google's agent-execution platform. Sandbox gives each agent a low-latency, gVisor-isolated environment with an integrated warm pool; Substrate multiplexes millions of short-lived "actors" onto pre-warmed pods with pod-snapshot suspend/resume.
- **Actor / WorkerPool / ActorTemplate** — an *actor* is one agent session. `ActorTemplate` is the per-actor workload spec; `WorkerPool` is the pool of warm pods actors are multiplexed onto (an **N:1** relationship). Substrate's group is `ate.dev` (not `agentsubstrate.google.com`, as earlier thread comments assumed).
- **gVisor / Sentry / runsc** — gVisor is a userspace kernel. `runsc` is its runtime; the **Sentry** intercepts and services the sandboxed process's syscalls in userspace, so they never reach the host kernel.

- **seccheck** — gVisor's first-class trace subsystem. Its `remote` sink serializes each trace *point* as a protobuf and streams it over a `SOCK_SEQPACKET` Unix socket to an external monitor. `runsc trace create` attaches a session to an already-running sandbox.
- **node-agent** — Kubescape's DaemonSet runtime sensor. It derives exec/network/file signals from host-kernel eBPF (Inspektor Gadget) and maintains an *Application Profile* behavioral baseline per workload.
- **CADR** — Cloud Application Detection & Response: the market category for runtime, behavior-based detection that ARMO/Kubescape and peers are moving toward, distinct from posture-only CSPM/KSPM.

Current state of the world. Kubescape scans standard workloads (core/apps/batch API groups) statically and, via node-agent, at runtime. Two agent controls (C-0297, C-0301) are merged into regolibary but belong to **no framework**, so they run in no scan (their PRs explicitly left framework registration out of scope). No agent CRD is a scan target. The node-agent has no path into a gVisor sandbox. Two adjacent efforts exist upstream: [node-agent#894](#) identifies seccheck as a Part B signal source and names its single-session limitation (note-only; no consumer code), and [regolibary#761](#) (Aug 4, draft) proposed a broader *AgentRuntimeHardening* control set — parked at maintainer request ("let's wait anyway to avoid any interference") pending this LFX project.

Goals and Non-Goals

Goals

- Treat agent-runtime CRDs as first-class posture + admission targets (Part A).
- Prove and land a runtime-visibility signal source from inside gVisor into the node-agent pipeline (Part B).
- Map controls to OWASP-LLM-2025 and MITRE ATLAS for security-team legibility.
- Keep the whole thing cheap: node-local, out-of-band, offline-validatable.

Non-Goals

- No re-architecture of node-agent's host-eBPF sensing. Part B *adds* a source.
- No per-session behavioral *enforcement* inside the sandbox in v1 (detection first).
- No cross-object ratio checks (per-actor-vs-pool ceilings) in Part A v1 — deferred; Substrate's own #679 (right-sizing actor sandboxes from declared `ActorTemplate` limits) reduces the urgency of cross-object math.
- No enumeration of live actors from Substrate's side control plane — static CRD scanning covers the *definitions*, which is the correct v1 scope.

Product & User Use Cases

The proposal is only worth building if it maps to jobs real people are trying to do. Five use cases, by persona:

UC-1 — Platform / GKE security engineer: "block an unsafe agent before it runs." A team ships an `ActorTemplate` that runs on the default runtime (shared host kernel) with open egress. At author time (IDE/CI) and at deploy time (admission), Kubescape flags or rejects it: *isolation class is not gVisor/Kata; egress is not*

default-deny. This is the left-to-right guardrail Kubescape already tells for standard workloads, extended to agents. (FR-1, FR-2, FR-4)

UC-2 — SecOps / detection engineer: "see what a sandboxed agent actually did." An agent is prompt-injected into running `curl ... | sh`, connecting to an external host and reading `/etc/shadow`. Host eBPF sees nothing — it's all inside gVisor. The Part B sensor recovers the exec, the egress destination, and the sensitive-file open, attributes them to the actor's container id, and the *existing* runtime rules fire. This is the "50,000 records exfiltrated, every native GKE control passed" scenario ARMO describes, made visible. (FR-5, FR-6)

UC-3 — Compliance / GRC: "prove tenant isolation for multi-tenant agents." An auditor needs evidence that no single actor can exhaust a shared `WorkerPool` and that snapshot storage (volatile memory + FS state at rest) is encrypted and access-scoped. The framework's per-actor-ceiling and snapshot-encryption controls produce the finding + remediation trail, mapped to OWASP-LLM / ATLAS. (FR-2, FR-3)

UC-4 — Incident responder: "attribute an egress spike to a session." During an incident, the responder needs per-actor attribution of an outbound connection. Even where full syscall visibility is unavailable (a held `Default` session), the host-boundary fallback still gives session start/end and gofer-initiated egress, plus an explicit posture finding that visibility is degraded on that node — so the gap is known, not silent. (FR-7)

UC-5 — Cluster operator / FinOps: "turn this on without a telemetry bill shock." The operator enables the sensor and wants predictable cost. Node-local profiling persists only anomalous/first-seen events (~3–5%), keeping incremental infrastructure cost to ≈\$1/node/month storage and single-digit dollars of egress even at high actor density (cost model §). (NFR-1, NFR-5)

Gap Analysis

What is already captured upstream, and what does this RFC actually add? The honest framing first: **the white space is already recognized** — the blind spot, the posture/runtime split, the candidate signal sources, and the mapping goals are all on record in [#10](#), the

This RFC's contribution is **execution, empirical validation, and integration design**: converting the recognized problem into a Kubescape-shaped design and a working implementation. Below is the line-by-line reconciliation against what a production-grade agent-security capability needs.

Captured officially (in #10 / #2557)

- Agent CRDs as scan targets; the six candidate control families (isolation, egress, ceilings, provenance, credential scoping, snapshot storage). ✓
- Admission variant (VAP / Kubescape admission). ✓
- OWASP-LLM / MITRE ATLAS mapping intent. ✓
- The N:1 `WorkerPool` ↔ actor posture problem, and the CREATE/UPDATE-vs-spawn admission scope caveat (raised by @Aryanbhargava18, deepened by @Shivampal157). ✓
- The gVisor visibility gap acknowledged and *explicitly deferred* to separate work; node-agent#894 independently identifies **seccheck** (remote sink, reference server, dynamic attach via `runsc trace create`) as the strongest Part B source **and names the single-`Default`-session limitation**. ✓

- Trace-stream schema-stability risk and alternatives raised in-thread — Falco's gVisor-engine removal, host-boundary signals as the stable-but-coarse path, the `runsc` metric-server endpoint as a middle path (@Shivampal157). ✓
- A broader *AgentRuntimeHardening* control set already proposed in regolib#761 (Aug 4, draft) — parked by maintainers to avoid interference with this LFX project; the merged C-0297/C-0301 PRs left framework registration explicitly out of scope. ✓
- CEL/VAP plumbing for custom apiGroups already landing via normal contributions (#3177 plural resolution, #3196 version discovery, #3099 `request.resource`). ✓

What this RFC adds on top of the prior discussion

#	Where the discussion stands	This RFC's addition
G-1	CRD identity was guessed wrong in-thread (comments assumed <code>agentsubstrate.google.com</code>), and the stability <i>concern</i> was raised without evidence.	The corrected identity map, pinned from upstream: Substrate is <code>ate.dev</code> (all v1alpha1); Sandbox is <code>agents.x-k8s.io</code> + <code>extensions.agents.x-k8s.io</code> (SandboxTemplate at v1beta1) — plus quantified churn evidence (37 manifest commits since mid-May; pause-image move #848, secretKeyRef removal #835, right-sizing #679) motivating field access behind a thin helper.
G-2	Signal source identified (#894 argues for seccheck) but no consumer exists — no fork, no code, nothing feeds node-agent.	A working PoC of the seccheck remote-sink path, decoding real gVisor protobufs into node-agent events, with a cluster-free E2E, a real- <code>runsc</code> path, and a measured performance envelope.
G-3	The single-<code>Default</code>-session constraint is named (#894) but not designed for. It is the crux of productionizing Part B.	Reproduced and designed around: contention detection → host-boundary fallback → degraded-visibility posture finding → per-node sink broker for multi-consumer sharing → upstream multi-session ask.
G-4	Schema-stability risk is raised in-thread (Falco's v0.44.0 gVisor-engine removal, metric-server alternative — @Shivampal157) but unmitigated.	A concrete, implemented mitigation: decode isolated behind bindings generated from the vendored <code>.proto</code> s with provenance, versioned; handshake + unknown-point tolerance per gVisor's own sink guidance; metric-server retained as the stable-but-coarse alternative.
G-5	No cost / performance envelope.	Measured benchmarks + a cost model (this doc).
G-6	No behavioral-baseline story for agents. Posture ≠ detection.	Part B feeds node-agent's existing Application Profile so per-agent behavioral baselining ("Application Profile DNA") applies to sandboxed actors — the ARMO-stated direction, extended into the sandbox.
G-7	Response/enforcement path unstated.	v1 is detection + admission; graduated response (network-policy soft-quarantine, seccomp tightening) named as future work, matching peer-vendor "observe → baseline → enforce."

Competitive & Vendor Landscape

Context for reviewers: what cybersecurity vendors typically offer for this class of problem, what they target, and where this proposal sits.

The market is moving from posture to runtime (CADR). The 2026 cloud-native detection-and-response field — Sysdig Secure (on open-source **Falco**), **Aqua**, Palo Alto **Prisma Cloud**, **Wiz**, **CrowdStrike** Falcon Cloud Security, Microsoft Defender for Containers, and **ARMO CADR** (Kubescape's commercial parent) — competes on a common capability stack:

- **eBPF kernel-level sensing** with **per-workload behavioral baselines** ("learn expected behavior for every workload").
- **Full-stack correlation** (app ↔ container ↔ K8s ↔ cloud) that produces "attack stories" rather than isolated alerts.
- **Admission + graduated response** beyond kill/isolate — soft-quarantine via network policy, seccomp tightening, drift detection.
- **Low overhead as a selling point** — vendors cite **1–2.5% CPU** as competitive.

What these solutions target (the threat taxonomy this proposal must speak to): reverse shells and RCE, application-layer attacks (SSRF, command/SQL injection), lateral movement via stolen credentials, Kubernetes API abuse and **service-account token theft**, cloud escalation (IAM abuse, **S3/bucket exfiltration**), supply-chain compromise, and cryptomining.

Pricing model. Per-node and/or per-workload licensing, with runtime frequently a separate add-on module (Prisma). This matters for the cost framing: the vendor's own list price sits on top of the infrastructure cost this RFC models.

Where this proposal sits — the differentiated wedge. Every vendor above assumes its eBPF sensor can see the workload. **For gVisor-isolated agents, that assumption breaks** — which is exactly why Falco *removed* its gVisor engine rather than maintain it. This proposal:

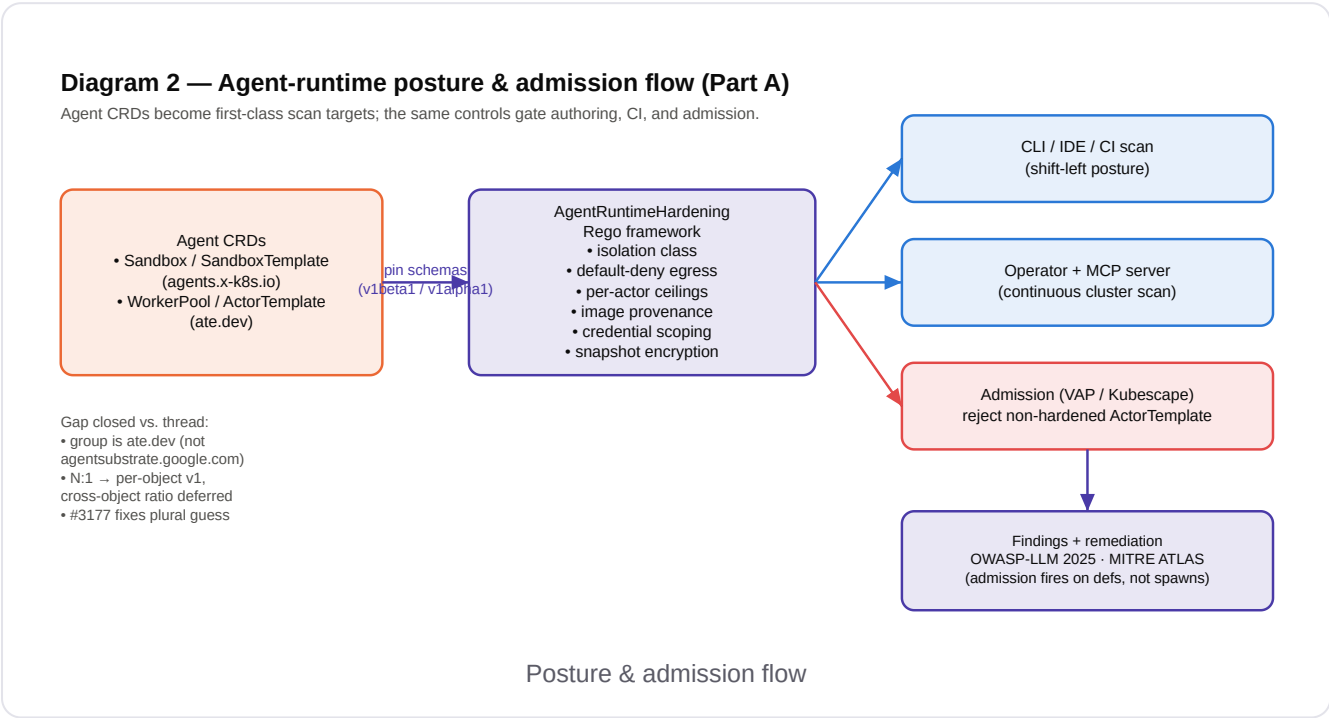
1. Is the **only open, upstream** attempt to restore that visibility for the specific agent-sandbox case, feeding a CNCF sensor (node-agent) that **100,000+ organizations** already run via Kubescape.
2. Attacks the **agent-specific** threats the generic taxonomy misses: prompt-injection → excessive-agency egress, per-session credential blast radius, N:1 tenant starvation, snapshot-at-rest exposure — mapped to **OWASP-LLM-2025** and **MITRE ATLAS**, not just the classic CNAPP kill-chain.
3. Keeps the classic protections (egress, lateral movement, token theft) by mapping the recovered signals onto node-agent's *existing* rules, so no detection content is rewritten.

In short: the incumbents target the runtime; none of them can currently see the agent runtime. That is the gap.

Design

The design has two independently shippable halves. Diagrams are provided as editable Excalidraw sources (`diagrams/*.excalidraw`) and rendered SVGs.

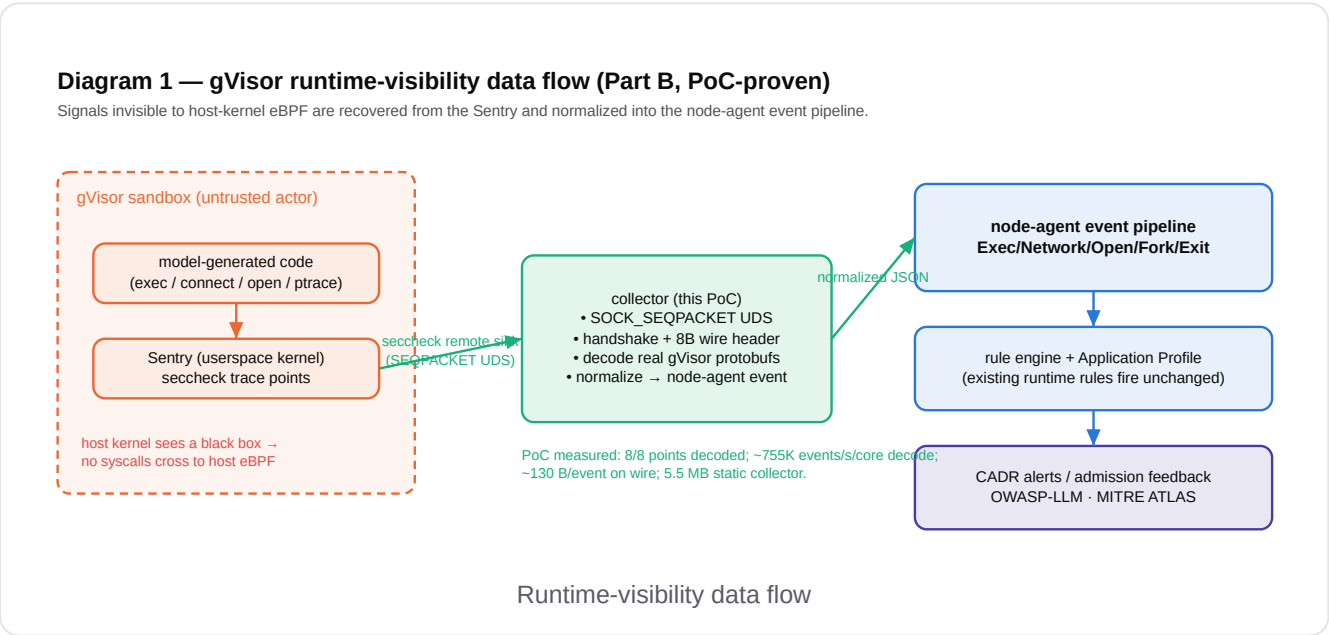
Part A — posture & admission



Control	Checks	OWASP-LLM 2025	MITRE ATLAS (tactic)
Snapshot storage protected	snapshot target encrypted + access-scoped	LLM02 Sensitive Info Disclosure	Collection

N:1 handling (the thread's open design gap). A `WorkerPool` sets pool-level compute; an `ActorTemplate` sets per-actor spec. v1 scopes to *per-object* checks (isolation, egress default, "limits exist") and explicitly defers cross-object ratio math (does the `ActorTemplate` request bound relative to `WorkerPool` capacity), which Rego cannot do cleanly unless both objects are in one scan batch. Documentation makes the admission-scope caveat explicit: the VAP fires on `ActorTemplate` CREATE/UPDATE, **not** on out-of-band actor spawns through Substrate's side control plane.

Part B — runtime visibility (PoC-proven)



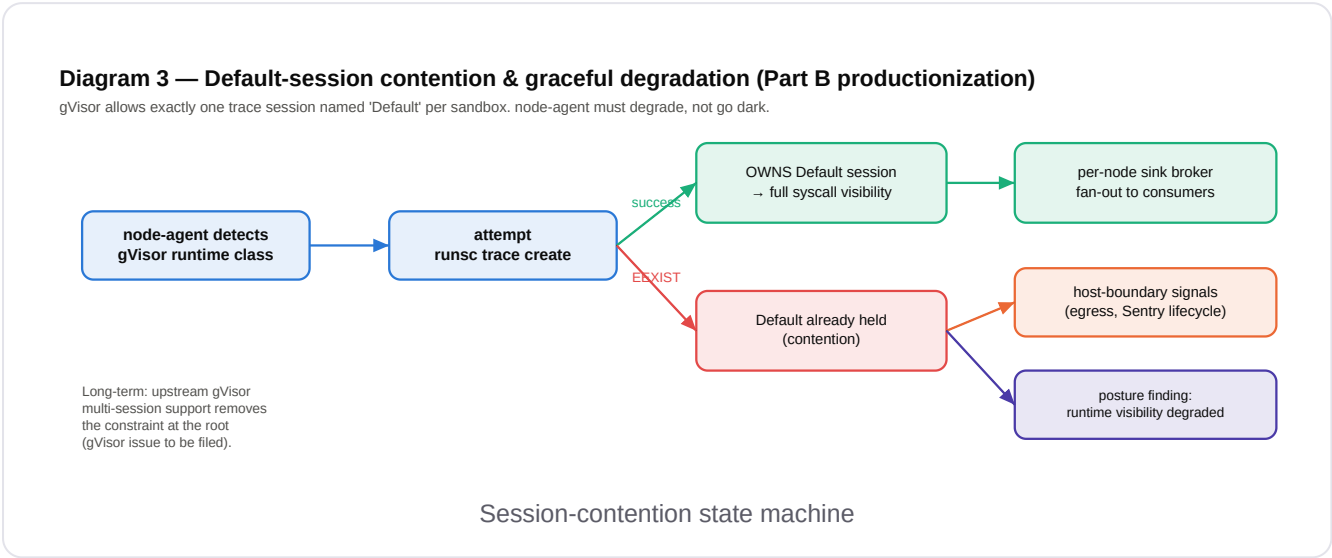
The node-agent derives runtime signals from host-kernel eBPF; under gVisor the sandboxed process traps into the Sentry and the host kernel sees almost nothing. The sensor recovers visibility from the Sentry side:

- `cmd/collector` binds the `SOCK_SEQPACKET` UDS, performs the seccheck handshake, reads the 8-byte-header + protobuf datagram stream, decodes each point against gVisor's **real** schemas (bindings generated from the vendored `.proto`), and normalizes it into the node-agent event shape.
- `internal/normalize` is the documented seccheck → node-agent mapping (exec/network/open/fork/exit/ptrace + container-start metadata for correlation).
- `cmd/fakesentry` + `make e2e` is a cluster-free E2E: a faithful replayer drives the *real* wire protocol through a scripted actor session (start → clone → exec `curl...|sh` → socket → connect to an external IP → open `/etc/shadow` → `ptrace` → exit). The collector decodes all 8 points and maps 7 (container-start is unmapped metadata, retained for correlation).
- `scripts/run-with-runsc.sh` is the real-gVisor path: `runsc trace create` a session against a running `--runtime=runsc` sandbox; identical collector code, only the sender differs.

node-agent integration seam. The normalized `Event` mirrors what a node-agent tracer callback assembles. Integration is: (1) wrap the collector as a `containerwatcher` tracer, gated on a gVisor runtime class; (2)

translate normalized events into node-agent's concrete `ExecEvent` / `NetworkEvent` / `OpenEvent` structs so existing rules and the Application Profile baseline fire unchanged; (3) correlate `container_id` to pod/actor via the mapping node-agent already maintains, seeded by the `MESSAGE_CONTAINER_START` point.

The `Default`-session constraint



gVisor permits **exactly one trace session per sandbox**, and it must be named `Default` ("only a single session can exist in the system and it must be called `Default`" — gVisor seccheck docs). If the platform or another tool already holds a sandbox's session, `trace create` fails. Note the asymmetry: collection does **not** need a broker — the remote sink's connection model already lets one process "monitor all sandboxes in the machine" (gVisor sink docs). The broker below solves the *downstream* problem: sharing each sandbox's single session across multiple consumers. The design: **detect** the contention (don't assume ownership) → **fall back** to host-boundary signals (egress at the CNI/conntrack layer, Sentry process lifecycle) so coverage degrades rather than going dark → **raise a posture finding** that runtime visibility is unavailable on that node (itself security-relevant) → medium-term, a **per-node sink broker** where node-agent owns the one session and re-publishes to multiple consumers → long-term, an **upstream gVisor** contribution for multi-session support (the constraint is flagged as removable in gVisor's own source comments).

Benchmarks & Performance

All PoC numbers below are measured in a Linux sandbox (Intel Xeon @ 2.10 GHz, Go 1.24.7, `go test -bench`, single core), not estimated. Cluster-scale rows are clearly labelled as estimates derived from those measurements.

Measured — decode + normalize (the collector hot path)

Point type	Latency	Throughput/core	Wire size	Allocs
<code>connect()</code> (network egress)	1,218 ns/op	~821K ev/s	78 B + 8 B hdr = 86 B	15 (912 B)

Point type	Latency	Throughput/core	Wire size	Allocs
<code>execve()</code> (process exec)	1,432 ns/op	~698K ev/s	166 B + 8 B hdr = 174 B	22 (1,184 B)
Blended	~1,325 ns/op	≈755K ev/s/core	~130 B/event	—

- **E2E correctness:** **PASS: 8/8 points decoded from the real seccheck wire format and normalized to node-agent events** — including recovering egress dst `203.0.113.7:80` from the raw `connect()` sockaddr and attributing every event to the sandbox id.
- **Collector binary:** 5.5 MB static, stdlib-only socket path (no cgo).

Estimated — cluster-scale, derived from the measured hot path

Load profile (per node)	Events/s/node	Decode CPU	UDS throughput
Low (50 actors, 20 ev/s, 15% active)	150	~0.0002 core	0.02 MB/s
Med (150 actors, 50 ev/s, 20% active)	1,500	~0.002 core	0.20 MB/s
High (300 actors, 100 ev/s, 25% active)	7,500	~0.010 core (0.12% of an 8-vCPU node)	0.97 MB/s

Even the High profile spends **~1% of a single core** on decode — well under the 1–2.5% CPU envelope competitors advertise for their *entire* sensor, because this is one out-of-band decode loop, not an inline interception path. *Events/s figures assume a security-relevant point subset (exec/net/open/fork/exit/ptrace), not every syscall — the seccheck session config selects points, which is the primary cost lever.*

Context — gVisor's own overhead (published, for completeness): 10–30% on I/O/syscall-heavy workloads, minimal for CPU-bound; the sensor adds nothing inline on top of this since it consumes the Sentry's existing trace stream.

Resource & Telemetry Cost Model

Incremental infra cost of running Part B, modeled from the measured hot path. Prices use GCP list (compute `n2-standard-8` ≈ \$0.388/hr; regional storage \$0.02/GB-mo; inter-region egress \$0.08/GB). Assumes node-local profiling persists **~3%** of events (anomalous/first-seen only, as node-agent's Application Profile already does).

Cost component (per node / month)	Low	Med	High
Decode compute	\$0.01	\$0.07	\$0.35
Hot storage (7-day, post-profiling)	\$0.02	\$0.21	\$1.03

Cost component (per node / month)	Low	Med	High
Egress to central SIEM (3% shipped)	\$0.35	\$3.55	\$17.73
Total / node / month	\$0.38	\$3.83	\$19.11

The headline cost finding: at every load level, **egress dominates** — at High load it is ~50× the compute cost. A 100-node cluster at High load is **≈\$1,912/month incremental, of which ~\$1,773 is egress**. This is the architectural argument for **node-local profiling and aggregation before export**: shipping raw normalized JSON (246 GB/node/day at High — the unprofiled upper bound) is untenable, but persisting only the profiled 3% makes the capability essentially free on compute and cheap on storage. The egress row assumes inter-region/SIEM shipping at GCP list rates; same-region or internet egress prices differ, but the ordering (egress >> storage >> compute) is robust across realistic rates. Design decisions (what to persist, where to profile, whether to keep events node-local vs. centralize) should be driven by this number, not by the negligible decode cost.

Timeline

12-week LFX term (Sep 7 – Nov 27), scoped to complement in-flight contributions.

Weeks	Milestone
1–2	Drive the opened framework PR (regolibrary#789 , sequenced after the C-0301 renumber #784) and the design PR (designs-and-proposals#14) through review — or park them until maintainers prefer, as with #761; reconcile the control set with #761; pin CRD fixtures (<code>extensions.agents.x-k8s.io/v1beta1</code> ; <code>ate.dev/v1alpha1</code> behind a helper).
3–5	Part A complete for Agent Sandbox (1:1); Substrate per-object controls with N:1 documented; confirm ATLAS technique IDs with maintainers.
6–8	Part B into node-agent: wrap collector as a <code>containerwatcher</code> tracer gated on gVisor runtime class; translate to concrete event structs; seed attribution.
9–10	Session-contention: <code>Default</code> -ownership detection, host-boundary fallback, degraded-visibility posture finding; file the upstream gVisor multi-session issue.
11–12	Admission variant on the merged CEL/VAP plumbing; e2e on a real GKE Agent Sandbox cluster; docs, demo, remediation text.

Dependencies

- **Internal:** regolibary (frameworks + controls), kubescape core (scan + CEL/VAP admission; #3177/#3196/#3099 plumbing already merged), node-agent (`containerwatcher`), Application Profile, reporters/rule engine), helm-charts (opt-in RBAC for agent runtime posture, [helm-charts#893](#)).
- **External:** gVisor (`runsc` (`seccheck` remote sink, `runsc trace create`); Agent Sandbox (`kubernetes-sigs/agent-sandbox`) and Agent Substrate (`agent-substrate/substrate`) CRD schemas; GKE Agent Sandbox cluster for term-time e2e; protobuf runtime.

Alternatives Considered / Prior Art

Approach	Verdict
Do nothing; wait for the runtime work	Rejected. Part A ships in a quarter and is useful immediately; it does not block on Part B.
Generic "AI security" checks on existing frameworks	Rejected. The agent CRDs have specific, checkable fields; vague checks don't survive review.
<code>runsc --strace</code> / <code>debug-log export</code> (Part B)	Dev-only. It's a text debug format, lossy under load, not a stable API.
Host-boundary signals only (Part B)	Kept as the <i>fallback</i> , not the primary — always-available but coarse (no per-syscall detail).
<code>seccheck</code> remote sink (Part B)	Chosen primary. Structured, typed, dynamically attachable, cheap to consume — and proven by the PoC.
Falco's gVisor engine	Prior art, now <i>removed</i> (v0.44.0, May 2026) over trace-proto schema-stability burden — validating both the demand and the need to isolate the decode behind generated bindings.
gVisor <code>tracereplay</code> (save/replay of remote-sink sessions)	Prior art for generic replay — it exists upstream. <code>fakesentry</code> differs in what it asserts: the <code>seccheck</code> → node-agent normalization contract (wire message → decode → expected exec/network/open semantics), not protocol replay per se.
node-agent#894 (seccheck design note)	Identifies <code>seccheck</code> (remote sink, reference server, dynamic attach) and names the single- <code>Default</code> -session limit — note-only, no consumer code. This RFC's PoC is the executable version, and the contention/broker design is the operational answer to the limit #894 records.
regolibary#761 (broader <i>AgentRuntimeHardening</i> controls, Aug 4, draft)	Prior art for Part A — parked at maintainer request to avoid LFX interference. This RFC's framework (#789) is the <i>minimal</i> wiring of already-merged controls; the term plan reconciles with #761's larger control set rather than duplicating it.

Operations

- **Who runs it:** the sensor ships as part of the node-agent DaemonSet; no new standing human process. Rego controls run in existing scan/operator/admission paths.
 - **Telemetry & metrics to collect:** per-node points-decoded / mapped / unmapped / dropped (the collector already emits these), `Default`-session ownership state, degraded-visibility node count, and the persisted-event ratio (the main cost driver).
 - **Rollout:** opt-in (a `--substrate`-style flag or a distinctly named bundle) so clusters without Agent Sandbox/Substrate installed see zero-match no-ops rather than noise or binary bloat; RBAC is opt-in per helm-charts#893.
 - **New human process:** maintainers confirm specific MITRE ATLAS technique IDs (left unfabricated by design) and own the `ate.dev` field-path helper as the schema drifts.
-

Security / Privacy / Compliance

- **Trust model.** The seccheck signal is *Sentry-side*: it trusts the Sentry (the isolation boundary), not the sandboxed workload. gVisor's own sink documentation makes the collector's posture mandatory: *"the monitoring process must validate and never trust input received from the Sentry because it can be controlled by a malicious user. All fields must have hard coded size limits."* The collector therefore: (1) enforces hard-coded per-field size caps; (2) tolerates unknown point types and versions — the connect-time handshake negotiates compatibility, and unknown protos are skipped and counted, never fatal; (3) accounts dropped messages via the wire header's control fields, so shedding is observable; (4) isolates each sandbox's connection with bounded per-connection buffers and shed-with-counters backpressure, so one hostile or chatty actor cannot starve other sandboxes' streams or block the Sentry.
 - **Sensitive data.** Recovered events can contain sensitive fields (argv, file paths, destinations, and — if enabled — execve binary hashes). Binary hashing and full-argv capture are **opt-in**; sensitive fields should be maskable consistent with any PII-masking policy before persistence or export. No secrets are added to the data model by Part A.
 - **Compliance value.** The framework's OWASP-LLM / ATLAS mapping and the snapshot-encryption / credential-scoping controls are the evidence GRC teams need for multi-tenant agent isolation (UC-3).
 - **Least privilege.** Agent-runtime RBAC is opt-in (helm-charts#893), scoped to the agent CRD groups.
-

Risks

Risk	Likelihood	Mitigation
CRD schema drift (<code>ate.dev/v1alpha1</code> still moving — 37 commits since May)	High	Field access behind a thin helper updatable in one place; pin Sandbox against the more-stable v1beta1.
Trace-point proto instability (the reason Falco dropped gVisor)	Med	Generated bindings from vendored <code>.proto</code> with provenance; metric-server endpoint as a stable-but-coarse alternative; version the decode.
Default-session contention with GKE-managed tooling	Med	Detect + degrade + posture finding; needs a real GKE cluster to settle ownership policy (term-time).
Admission scope misunderstood (fires on defs, not spawns)	Med	Explicit control-description + remediation text; documented as a known boundary.
Competitive pre-emption (a vendor ships gVisor visibility first)	Low-Med	Open, upstream, CNCF-sensor-native, and already de-risked by the PoC.

Impact on cluster / node-agent performance

The added path is one out-of-band decode loop measured at **~0.12% of an 8-vCPU node** even at 7,500 events/s; it never blocks the actor's hot path. **Measured before production** via the `go test -bench` suite in the PoC and the per-node decode/drop counters. If overhead materializes (e.g., a misconfigured session enabling every syscall point), mitigations are: narrow the seccheck point set, sample, and cap the collector's CPU via the DaemonSet resource limits — the drop counter makes shedding observable rather than silent.

Future Work

- **Graduated response** (currently detection-only): network-policy soft-quarantine, per-actor seccomp tightening, admission-time rejection — the "observe → baseline → enforce" arc peers offer.
- **Cross-object ceiling ratios** (per-actor vs `WorkerPool` capacity) once multi-object scan batching supports it.
- **Live-actor enumeration** from Substrate's side control plane for runtime attribution beyond the CRD definitions.
- **Upstream gVisor multi-session support**, removing the `Default` constraint at the root.
- **Domain enrichment via existing DNS facilities**. A `connect()` point carries only the destination sockaddr (IP, port, address family) — not the hostname that led to it. Domain attribution therefore comes from correlating gVisor network events with DNS query/response telemetry and node-agent's existing DNS tracer + `dnsmanager` address → domain cache, where that relationship is observable — not from `connect()` alone. Query-level analytics (DGA entropy/length/frequency, DNS exfiltration) additionally require the DNS points themselves.

- **Fold under CADR:** land the sensor standalone (independently useful), reference it from the broader CADR design.
-

Audit Log

- **New audit entries:** admission decisions on agent CRDs (accept/reject/warn with the failing control), and degraded-visibility events (a `Default` session could not be acquired on node X) — both are security-relevant state changes worth an audit trail.
 - **Existing audit logging:** unaffected; Part A rides existing scan/admission audit paths.
-

References

- Issue: [kubescape#2557](#) · pinned CRD-identity comment: [#issuecomment-5314471897](#)
 - Design proposal: [designs-and-proposals#10](#) · LFX tracker: [cncf/mentoring#1991](#)
 - Merged plumbing: [#3177](#), [#3196](#), [#3099](#) · regolibary [#768/#778/#787/#784](#) · [helm-charts#893](#)
 - Adjacent prior art: [node-agent#894](#) (seccheck note) · [regolibary#761](#) (broader framework draft, parked) · this RFC's PRs: [regolibary#789](#), [designs-and-proposals#14](#)
 - gVisor sink/trace internals: [seccheck README](#) · [remote-sink README](#) ([untrusted-Sentry guidance](#)) · [tools/tracereplay](#)
 - Upstream CRDs: [kubernetes-sigs/agent-sandbox](#) · [agent-substrate/substrate](#) (#848, #835, #679)
 - Google: [Agent Sandbox on GKE + Agent Substrate](#) · [InfoQ coverage](#)
 - ARMO: [Sandboxing AI Agents on GKE — where gVisor/WLI/VPC-SC stop working](#) · [Best CADR solutions 2026](#)
 - gVisor: [seccheck / systrap](#) · [install](#)
 - OWASP Top 10 for LLM Applications 2025 · MITRE ATLAS
 - Companion PoC & benchmarks: `gvisor-visibility-poc` (E2E 8/8; `go test -bench` numbers in §Benchmarks) — public repository link to be added on publication
-

Prepared for review by the Kubescape maintainers · upstream: [kubescape#2557](#) · [designs-and-proposals#10](#) / [#14](#) · [regolibary#789](#) · [cncf/mentoring#1991](#). Diagrams are inlined SVG; editable Excalidraw sources available on request.